Thema:

08 Docker Compose

Einleitung

In diesem Kapitel werden wir uns mit dem Container-Management-Tool Docker Compose befassen. Docker Compose ist eine Möglichkeit, mehrere Docker-Container gleichzeitig zu erstellen und zu verwalten. Es ermöglicht die Erstellung komplexer Anwendungen aus einzelnen Mikroservices und bietet eine einfache und intuitive Oberfläche für die Konfiguration von Konten.

ⓘ Lernziele

- Das Verständnis für die Grundlagen von Docker Compose soll aufbaut werden.
- Die Fähigkeit erlangen, komplexe Anwendungen mit Docker Compose zu erstellen.
- Kenntnisse über die Verwaltung von Konten in Docker Compose aufbauen.

Einführung in Docker Compose

Vorteile von Docker Compose

Docker Compose ermöglicht die Erstellung von komplexen Anwendungen aus einzelnen Mikroservices, was die Skalierbarkeit, Flexibilität und Wartungsfreundlichkeit erhöht.

Es bietet eine einfache und intuitive Oberfläche für die Konfiguration und Verwaltung von Konten, einschliesslich der Definition von Servieren, Ports und Netzwerken.

Durch die Möglichkeit, mehrere Container gleichzeitig zu erstellen und zu verwalten, kann Docker Compose die Skalierbarkeit von Anwendungen verbessern und sie besser auf Veränderungen in der Anforderung anpassen. Durch die Möglichkeit, Container automatisch zu starten, stoppen und zu restarten, kann Docker Compose die Wartung von Anwendungen erleichtern und Fehler minimieren.

Docker Compose bietet eine flexible Konfigurationsmöglichkeit, die es ermöglicht, Anwendungen unter verschiedenen Umgebungsbedingungen zu konfigurieren.

Grundlagen von Docker Compose

Docker Compose besteht aus einer YAML-Datei, das eine Anwendung definiert. YAML (YAML Ain't Markup Language) ist eine benutzerfreundliche Daten-Serialisierungssprache, die hauptsächlich für Konfigurationsdateien verwendet wird. Die Syntax ist so gestaltet, dass sie sowohl für Menschen verständlich als auch für Computer optimiert ist.

Die `compose.yaml`-Dateien dienen nicht nur als Konfigurationsdateien für die Docker-Engine, sondern fungieren gleichzeitig als praktische Dokumentation über den Aufbau der Multi-Container-Anwendung. Dies macht es für Entwickler einfacher, die Struktur der Anwendung zu verstehen und zu warten.

Die standardmässige Compose-Datei wird als `compose.yaml` oder `compose.yml` im Arbeitsverzeichnis platziert, wobei auch die älteren Namen `docker-compose.yaml` und `docker-compose.yml` für Abwärtskompatibilität unterstützt werden.

Die Definition der YAML-Datei besteht aus einer Liste von Services, die mit einem bestimmten Port konfiguriert werden.

⚠ Beachte:

YAML-Dateien werden nur korrekt von Docker Compose eingelesen, wenn sie richtige Einrückung mit der Tabulator-Taste haben. Achten Sie darauf, sonst kann Docker Compose nicht ausgeführt werden.

ⓘ Zusammengefasst:

Docker Compose ermöglicht die Definition und Ausführung von Multi-Container-Anwendungen, wobei alle Dienste in einer einzigen YAML-Datei konfiguriert werden. Dies automatisiert die Verwaltung der Container und macht es möglich, mit nur einem Befehl sämtliche Dienste zu erstellen und zu starten.

Erstellung einer Docker-Composition

Services (Dienste)

Ein Service in Docker Compose repräsentiert eine grundlegende Komponente der Container-Orchestrierung und erfüllt mehrere wichtige Funktionen:

- Services sind abstrakte Definitionen von Computerressourcen innerhalb einer Anwendung, die unabhängig von anderen Komponenten skaliert oder ersetzt werden können. Sie werden durch Container unterstützt, die von der Plattform gemäss den Replikationsanforderungen und Platzierungsbeschränkungen ausgeführt werden.
- Jeder Service kann aus einem oder mehreren identischen Containern bestehen, die alle mit denselben Argumenten erstellt werden. Die Container innerhalb eines Services werden auf Basis eines Docker-Images und einer Reihe von Laufzeitargumenten definiert.
- Services können über ihre Namen miteinander kommunizieren, da Docker Compose automatisch DNS-Namen für die Services bereitstellt. Dies vereinfacht die Kommunikation zwischen verschiedenen Containern erheblich, da keine IP-Adressen verwendet werden müssen.

Netzwerke

Netzwerke in Docker Compose spielen eine zentrale Rolle für die Container-Kommunikation und erfüllen mehrere wichtige Aufgaben:

- Netzwerke ermöglichen die Kommunikation zwischen verschiedenen Containern innerhalb einer Docker Compose-Umgebung. Sie stellen sicher, dass Container sich gegenseitig erreichen können, während sie gleichzeitig voneinander isoliert bleiben.
- Docker Compose erstellt automatisch ein Standard-Netzwerk für alle Services in der Compose-Datei. Innerhalb dieses Netzwerks können Container sich über ihre Service-Namen gegenseitig finden, da Docker einen eingebauten DNS-Server bereitstellt.
- Durch die Verwendung verschiedener Netzwerke können Container-Gruppen voneinander isoliert werden. Dies erhöht die Sicherheit, da nur Container im gleichen

Netzwerk miteinander kommunizieren können.

Zwei wichtige Netzwerktypen existieren in Docker Compose:

- Bridge-Netzwerke für die Kommunikation zwischen Containern auf demselben Docker-Host
- Host-Netzwerke für direkten Zugriff auf das Hostsystem-Netzwerk

Volumes

Volumes in Docker Compose erfüllen mehrere wichtige Funktionen für die Datenspeicherung und den Datenaustausch:

- Volumes speichern Daten ausserhalb der Container und bleiben auch dann bestehen, wenn Container gestoppt oder gelöscht werden. Das kennen wir bereits aus dem Erstellen eines einzelnen Docker-Containers. Dies ist besonders wichtig für Datenbanken, Konfigurationsdateien und andere wichtige Anwendungsdaten, die über Container-Neustarts hinweg erhalten bleiben müssen.
- Volumes ermöglichen den Datenaustausch zwischen verschiedenen Containern. Mehrere Container können auf dasselbe Volume zugreifen und dadurch Daten miteinander teilen, was die Zusammenarbeit zwischen verschiedenen Diensten ermöglicht.
- Im Vergleich zu anderen Datenspeichermethoden bieten Volumes bessere Lese- und Schreibgeschwindigkeiten. Sie ermöglichen direkten Zugriff auf das Hostsystem-Dateisystem.

Beispiel für eine Docker-Composition

```yaml
version: '3'

services:
  web:
    build: .
    ports:
      - "8080:80"
    depends_on:
      - db
    environment:
      - DATABASE_URL=postgres://user:password@db:5432/database

  db:
    image: postgres
    environment:
      - POSTGRES_USER=user
```

```
      - POSTGRES_PASSWORD=password
      - POSTGRES_DB=database
```

Erklärung dieser YAML-Datei:

- **`version: '3'`** Diese Zeile gibt die Version der Docker Compose-Syntax an, das in diesem Beispiel verwendet wird. Dies ist in neuen Docker Compose Versionen nicht mehr nötig.
- **`services:`** Diese Zeile beginnt den Bereich für die Definition von Diensten (Service) in der Konfiguration.
- **`web:`** definiert einen Dienst namens "web".
- **`build: .`** Anstatt ein fertiges Image zu laden, sagt dieser Befehl Docker: „Schau in das aktuelle Verzeichnis, nimm das dortige `Dockerfile` und baue daraus ein eigenes Image.“
- **`ports:`** definiert eine Port-Konfiguration für den Web-Dienst
- **`8080:80`** öffnet Port 8080 im Host-System und remappt es auf Port 80 im Conatiner-System.
- **`depends_on:`** gibt an, dass der Web-Dienst auf den Datenbank-Dienst (db) abhängt und erst dann startet, wenn dieser Dienst gestartet ist.
- **`environment:`** definiert Umgebungsvariable für den Web-Dienst:
- **`DATABASE_URL=postgres://user:password@db:5432/database`** setzt eine Umgebungsvariable namens `DATABASE_URL`, die die Verbindungsdaten zum Datenbank-Server enthält.

Datenbank-Definition

- **`db:`** definiert einen Dienst namens "db".
- **`image: postgres`** gibt an, dass der Docker-Image für diesen Dienst die `postgres`-Basis-Image verwendet wird.
- **`environment:`** definiert Umgebungsvariable für den Datenbank-Dienst:
- `POSTGRES_USER=user`: Diese Zeile setzt eine Umgebungsvariable namens `POSTGRES_USER`, die den Benutzernamen für den Datenbank-Server enthält.
- `POSTGRES_PASSWORD=password`: Diese Zeile setzt eine Umgebungsvariable namens `POSTGRES_PASSWORD`, die das Passwort für den Datenbank-Server enthält.

Diese YAML-Datei definiert eine Web-Anwendung, die auf einem Datenbank-Server (PostgreSQL) abhängt und die Verbindungsdaten zum Datenbank-Server durch Umgebungsvariable definiert. Der Docker-Image für den Datenbank-Server wird von der `postgres`-Basis-Image verwendet.

Verwaltung von Konten in Docker Compose

- Die `docker-compose up`-Kommando startet alle Dienste in der Anwendung
- Die `docker-compose down`-Kommando stoppt alle Dienste in der Anwendung
- Die `docker-compose restart`-Kommando startet alle Dienste in der Anwendung neu

Die Anwendung dazu wird später im Fallbeispiel 2 gezeigt.

Übung und Praxis

Fallbeispiel 1:

Ohne Docker Compose

Das Beispiel soll zeigen, wie man ohne Compose 2 Container erstellt und diese über ein Netwerk miteinander kommunizieren lässt. Wir gehen folgendermassen vor:

- Ein Docker-Netzwerk erstellen
- Ein MongoDB Container erstellen
- Ein Mongo Express-Container erstellen (Ein User-Interface für MongoDB)

Netzwerk erstellen:

```
docker network create mongo-network
```

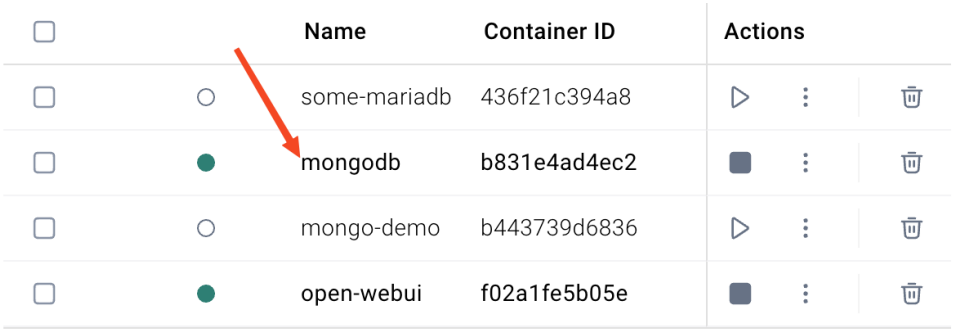
Netzwerk aufzeigen:

```
docker network ls
```

Container 1 erstellen (MongoDB):

```
docker run -d -p 27017:27017 -e MONGO_INITDB_ROOT_USERNAME=admin -e
MONGO_INITDB_ROOT_PASSWORD=password --name mongodb --net mongo-network
mongo
```

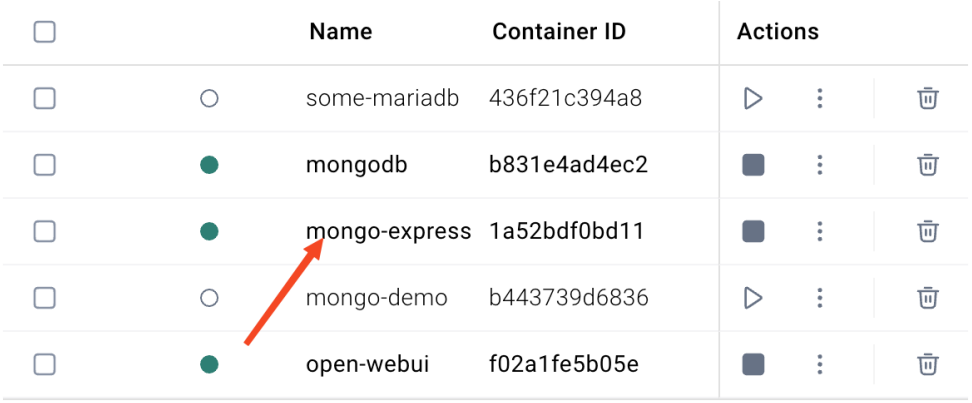
Hier werden dabei Umgebungsvariablen mitgegeben.



Container 2 erstellen (Mongo Express):

```
docker run -d -p 8081:8081 -e ME_CONFIG_MONGODB_ADMINUSERNAME=admin -e
ME_CONFIG_MONGODB_ADMINPASSWORD=password --net mongo-network --name
mongo-express -e ME_CONFIG_MONGODB_SERVER=mongodb mongo-express
```

Hier werden dabei Umgebungsvariablen mitgegeben.

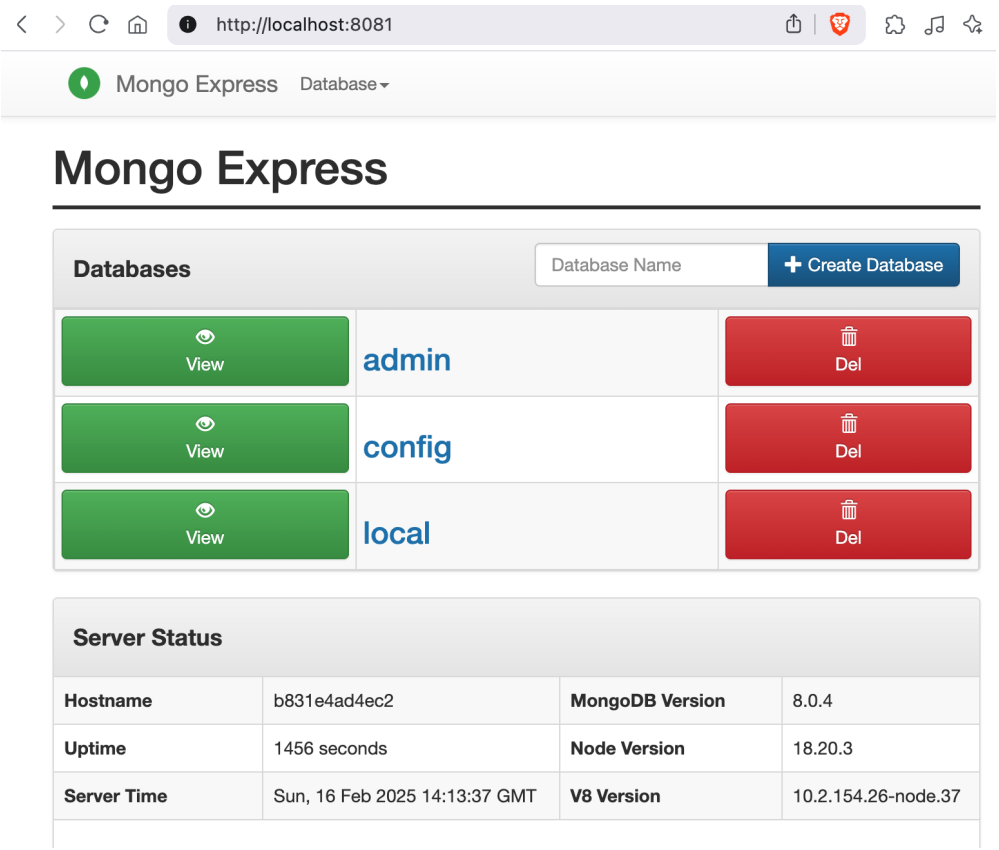


> ⚠ **Beachte:**
>
> das Erstellen eines Docker-Netzwerks ist optional. Sie können beide Container in einem Standardnetzwerk starten. In diesem Fall geben Sie einfach das `--net` Flag im `docker run` Befehl aus.

Testen der Applikation mit 2 Containern:
Im Browser geben wir folgende Adresse ein:

```
http://localhost:8081/
```

`Nutzername: admin`
`Passwort: pass`



Dieses zeigt wie Container, die zueinander gehören einzeln gestartet werden.

Frage:

Was ist, wenn wir nicht wie beim oberen Beispiel nur 2 Container haben, sondern z.B.

- 16 Microservices
- Messaging Service
- mehrere Datenbanken
- User Interface für alle Datenbanken

Dies sind viele Container und Services auf einmal und man kann hier schnell den Überblick verlieren.

Fallbeispiel 2:

mit Docker Compose

An dieser Stelle wollen wir das Gleiche bewerkstelligen, jedoch aus Gründen der Übersichtlichkeit packen wir alles in eine Docker Compose YAML-Datei.

Aus Docker run entsteht eine Compose-Datei:

```
# für MongoDB:
```

```
# docker run -d \
# -p 27017:27017 \
# -e MONGO_INITDB_ROOT_USERNAME=admin \
# -e MONGO_INITDB_ROOT_PASSWORD=password \
# --name mongodb \
# --net mongo-network \
# mongo

# für MongoExpress:

# docker run -d \
# -p 8081:8081 \
# -e ME_CONFIG_MONGODB_ADMINUSERNAME=admin \
# -e ME_CONFIG_MONGO_ADMINPASSWORD=password \
# -e ME_CONFIG_MONGODB_SERVER=mongodb \
# --name mongo-express \
# --net mongo-network \
# mongo-express

# Docker Compose-Inhalt:
version: '3'
services:

  mongodb:
    image: mongo
    ports:
      - 27017:27017
    environment:
      MONGO_INITDB_ROOT_USERNAME: admin
      MONGO_INITDB_ROOT_PASSWORD: supergeheim

  mongo-express:
    image: mongo-express
    ports:
      - 8081:8081
    environment:
      ME_CONFIG_MONGODB_ADMINUSERNAME: admin
      ME_CONFIG_MONGODB_ADMINPASSWORD: supergeheim
      ME_CONFIG_MONGODB_SERVER: mongodb
```

Wir sehen hier, dass bei der YAML-Datei kein Netzwerk angegeben wird. Docker erzweugt dabei ein eigenes Standard-Netzwerk.

Testen der Compose-Datei:

Zuerst müssen wir aufräumen. Die vorgehenden Container und Images werden entfernt, wie auch das von uns erstellte Netzwerk:

```
docker ps              # Anzeigen aller laufenden Container mit ihren IDs
docker stop ID1 ID2    # Stoppen der beiden Container
docker rm ID1 ID2      # Löschen der beiden Container
docker network rm mongo-network # Löschen des Netwerkes
docker network ls      # Anzeigen aller Netwerke
```

Starten der Compose-Datei:
Unsere oben in einem beliebigen Verzeichniserstellte YAML-Datei, nennen wir sie `mongo-services.yaml` wird mit folgenden Befehl gestartet:

```
docker-compose -f mongo-services.yaml up -d
```

Jetzt werden die definierten Container im detached Mode zusammengebaut, erstellt und gestartet.

Kontrollieren der richtigen Reihenfolge:

Um zu gewährleisten, dass die Abhängigkeiten korrekt sind, also MongoDB vor Mongo Express gestartet wird, muss dies in der Compose-Datei mitgeteilt werden:

```
  mongo-express:
    image: mongo-express
    ports:
         - 8081:8081
    environment:
        ME_CONFIG_MONGODB_ADMINUSERNAME: admin
        ME_CONFIG_MONGODB_ADMINPASSWORD: supergeheim
        ME_CONFIG_MONGODB_SERVER: mongodb
    depends_on:
        - "mongodb"
```

Löschen aller Container:

Mit diesem Befehl werden die in der Compose-Datei beschriebenen Containers gestoppt und **gelöscht**.

```
docker-compose -f mongo-services.yaml down
```

Stoppen und neustarten der Containers:

```
docker-compose -f mongo-services.yaml stop
docker-compose -f mongo-services.yaml start
```

Hier werden die Daten nicht entfernt.

Auftrag 1

Spielen Sie das Beispiel mit MongoDB und Mongo Express zuerst individuell mit `docker run` und danach mit mit Docker Compose durch und erstellen Sie eine Collection "PC" mit 2 Einträgen.

Fazit

Docker Compose ist ein mächtiges Werkzeug für die Erstellung komplexer Anwendungen aus einzelnen Mikroservices.

Gemeinsames Netzwerk in Docker Compose

Docker Compose ermöglicht das Erstellen eines gemeinsamen Netzwerks zwischen den Diensten, indem wir die `networks`-Einstellung in der docker-compose.yml-Datei verwenden.

Hier sind einige Beispiele:

Einzelnes Netzwerk für jedes Service:

```yaml
version: '3'
services:
  web:
    build: .
    ports:
      - "8080:80"
    networks:
      - mynet

  db:
    image: postgres
    environment:
      - POSTGRES_USER=user
      - POSTGRES_PASSWORD=password
      - POSTGRES_DB=database
    networks:
      - mynet

networks:
  mynet:
    driver: bridge
```

In diesem Beispiel definieren wir ein Netzwerk namens `mynet` und fügen es zu beiden Diensten hinzu. Jeder Service kann dann über das Netzwerk kommunizieren.

Ein Netzwerk für alle Dienste:

```yaml
version: '3'
services:
  web:
    build: .
    ports:
      - "8080:80"
    networks:
      - default

  db:
    image: postgres
    environment:
      - POSTGRES_USER=user
      - POSTGRES_PASSWORD=password
      - POSTGRES_DB=database
    networks:
      - default

networks:
  default:
    driver: bridge
```

In diesem Beispiel definieren wir ein Netzwerk namens `default` und fügen es zu beiden Diensten hinzu. Alle Dienste können dann über das Netzwerk kommunizieren.

Ein Netzwerk mit eigenen Subnetzen:

```yaml
version: '3'
services:
  web:
    build: .
    ports:
      - "8080:80"
    networks:
      - internal

  db:
    image: postgres
    environment:
      - POSTGRES_USER=user
      - POSTGRES_PASSWORD=password
```

```
      - POSTGRES_DB=database
    networks:
      - internal

networks:
  internal:
    driver: bridge
    ipam:
      - subnet: 10.0.1.0/24
```

In diesem Beispiel definieren wir ein Netzwerk namens `internal` mit einem eigenen Subnetz von `10.0.1.0/24`. Jeder Service kann dann über das Subnetz kommunizieren.

Docker Compose ermöglicht das Erstellen eines gemeinsamen Netzwerks zwischen den Diensten, indem wir die `networks`-Einstellung in der docker-compose.yml-Datei verwenden. Wir können ein Netzwerk für jedes Service erstellen oder ein Netzwerk für alle Dienste definieren und es mit Subnetzen konfigurieren, um sicherzustellen, dass die Kommunikation zwischen den Diensten ordnungsgemäss funktioniert.

Auftrag 2

Die Klasse wird in Gruppen von drei Personen aufgeteilt. Jede Gruppe sucht sich ein Container-Beispiel, dass mit Docker-Compose aufgebaut wird.
Die YAML-Datei wird aufgezeigt und erklärt und die Erstellung der Containers mit Compose wird ausgeführt. Danach wird die Arbeitsweise des Containers aufgezeigt.

Zeit: 90 Minuten
Präsentation: 5-10 Minuten

Fazit

Docker Compose ist ein mächtiges Werkzeug für die Erstellung komplexer Anwendungen aus einzelnen Mikroservices.

Wichtige Fragen zum Thema

Frage:

Was ist der Hauptvorteil von Docker Compose gegenüber normalen Docker-Befehlen?

Der Hauptvorteil von Docker Compose in der **Vereinfachung des Workflows** und der **strukturierten Verwaltung** von Anwendungen, die aus mehreren Containern bestehen.

Hier sind die wesentlichen Vorteile im Detail:

Zentrale Definition statt unübersichtlicher Befehle
Während man bei normalen Docker-Befehlen für jeden Dienst (z. B. Webserver, Datenbank, Backend) einen eigenen, oft langen und komplexen `docker run`-Befehl ausführen muss, wird bei Docker Compose die gesamte Anwendungsarchitektur in einer einzigen, lesbaren YAML-Datei definiert. Dies verhindert, dass der Prozess unübersichtlich wird, wenn man viele Parameter wie Ports oder Umgebungsvariablen verwalten muss.

Steuerung der gesamten Infrastruktur mit einem Befehl
Anstatt Befehle mehrfach schreiben zu müssen, können Sie mit einem einzigen Befehl die komplette Infrastruktur Ihrer Anwendung starten, stoppen oder verwalten. In der YAML-Datei werden alle Dienste und deren Kommunikation untereinander zentral festgelegt.

Umfassendes Aufräumen
Ein weiterer Vorteil gegenüber normalen Befehlen zeigt sich beim Beenden: Während `docker stop` einen Container nur anhält, entfernt der Befehl `docker compose down` nicht nur die Container, sondern auch das von Compose erstellte Netzwerk. Dies ermöglicht ein sauberes Aufräumen der gesamten Entwicklungsumgebung in einem Schritt.

Frage:

Wie sieht die Struktur einer typischen compose.yaml-Datei für einen Webserver aus?

1. Der services-Block
Jede Compose-Datei beginnt mit dem `services`-Block auf der obersten Ebene. Hier werden alle Container oder Dienste aufgelistet, aus denen die Anwendung besteht.

2. Die Dienst-Definition
Unterhalb von `services` wird der Dienst benannt (z. B. `webserver`). Dieser Name muss eingerückt sein (zwei oder vier Leerzeichen), um anzuzeigen, dass er zu den Services gehört.

3. Die Eigenschaften des Containers
Unter dem Namen des Dienstes werden die Eigenschaften definiert. Die Reihenfolge ist dabei nicht entscheidend, aber die Einrückung muss stimmen (auf derselben Ebene). Die wichtigsten Eigenschaften aus dem Tutorial sind:

`image` : Gibt das zu verwendende Docker-Image und dessen Version an (z. B. `nginx:1.29`), analog zum normalen `docker run` -Befehl.
`ports` : Definiert die Port-Weiterleitung (Mapping). Dies ist eine Liste, es können also mehrere Ports angegeben werden (entspricht dem `-p` Flag).
`container_name` : Legt einen festen, lesbaren Namen für den Container fest (entspricht dem `--name` Flag), damit er später leichter zu finden ist.
`restart` : Legt die Neustart-Richtlinie fest. Mit `unless-stopped` wird der Container automatisch gestartet, wenn Docker neu startet (z. B. nach einem Server-Reboot).

Beispiel-Struktur Basierend auf den Erklärungen im Video sieht eine korrekte Datei für einen Nginx-Webserver so aus:

```
services:
  webserver:
    image: nginx:1.29
    ports:
      - "8081:80"
    container_name: mein-webserver
    restart: unless-stopped
```

Wichtiger Hinweis zur Syntax: Die korrekte Einrückung ist entscheidend. Wenn beispielsweise `image:` falsch eingerückt wäre, würde der Editor oder Docker einen Fehler melden.

> **Frage:**
>
> Was ist der Unterschied zwischen `docker compose stop` und `down` ?

`docker compose stop` : Dieser Befehl **hält die Container lediglich an**. Die Container existieren weiterhin, befinden sich jedoch im Status "exited". Sie werden nicht gelöscht, und man kann sie später wieder starten (z. B. mit `docker compose start`).

`docker compose down` : Dieser Befehl führt ein komplettes Aufräumen durch. Er stoppt die Container nicht nur, sondern **entfernt sie vollständig**. Zusätzlich löscht er auch das **Netzwerk**, das Docker Compose für die Kommunikation zwischen den Diensten erstellt hat. Dies ist ideal, wenn man die Entwicklungsumgebung komplett bereinigen möchte.

> **Frage:**
>
> Welche Befehle helfen mir beim Debuggen, wenn ein Container nicht startet?

Die erste Anlaufstelle wäre der Befehl: `docker compose logs`

Wenn ein Container nicht startet oder abstürzt, sollten Sie laut Tutorial immer zuerst die Logs prüfen. Sie geben in der Regel einen direkten Hinweis auf das Problem (z. B. Konfigurationsfehler).

Hinweise mit dem Logfile auf Fehlersuche zu gehen:

Alle Logs anzeigen: `docker compose logs` zeigt die Protokolle aller Dienste an.
Logs eines bestimmten Dienstes: `docker compose logs [dienstname]` (z. B. `webserver2`) zeigt nur die Informationen für diesen spezifischen Container.
Live-Verfolgung (Follow): Mit dem Flag `-f` (oder `--follow`) können Sie die Logs in Echtzeit beobachten, während sie passieren. Dies ist hilfreich, um zu sehen, was passiert, wenn eine Anfrage an den Server gesendet wird.

Befehl: `docker compose logs -f [dienstname]`

Ausgabe begrenzen (Tail): Wenn die Logs zu lang sind, können Sie mit `--tail` nur die letzten Zeilen anzeigen lassen (z. B. die letzten 50), um schnell den aktuellen Fehler zu finden.

Befehl: `docker compose logs --tail=50`

Als zweite Variante kann man auch den Container prüfen: `docker compose exec`

Sollte der Container zwar starten, aber nicht wie erwartet funktionieren (z. B. falsche Konfiguration oder fehlende Dateien), können Sie Befehle direkt *innerhalb* des laufenden Containers ausführen.

Einzelne Befehle ausführen: Sie können überprüfen, ob Dateien vorhanden sind oder Konfigurationen stimmen, ohne in den Container hineingehen zu müssen.

- Beispiel: `docker compose exec webserver ls -l /etc/nginx/conf.d` prüft den Inhalt eines Konfigurationsordners.

Interaktive Shell starten: Für tiefgreifendere Prüfungen oder zum Bearbeiten von Dateien können Sie eine interaktive Sitzung starten.

- Befehl: `docker compose exec -it [dienstname] bash`.
  - *Hinweis:* Falls `bash` nicht installiert ist (bei kleineren Images), können Sie stattdessen `sh` verwenden.

Als dritte Variante kann man auch den Status des Containers prüfen: `docker container ls`

Um überhaupt zu sehen, ob ein Container läuft oder warum er beendet wurde, hilft der Listen-Befehl.
`docker container ls`: Zeigt alle laufenden Container an.
`ls -a`: Zeigt auch gestoppte Container an. Dies ist wichtig, wenn ein Container sofort nach dem Start wieder abgestürzt ist (Status "Exited")

Übersicht der Docker-Compose-Befehle und Konfigurationen

Befehl / Schlüsselwort	Kategorie	Beschreibung	Parameter / Optionen	Beispiel
docker compose up	Docker-Compose-Befehl	Erstellt und startet die in der YAML-Datei definierten Container.	-d (detached), -f (File-Pfad)	`sudo dock` `compose u`
docker compose down	Docker-Compose-Befehl	Stoppt und entfernt alle Container sowie die dazugehörigen Netzwerke.	Nicht in Quelle	`sudo dock` `compose d`
docker compose start	Docker-Compose-Befehl	Startet bereits existierende, gestoppte Container wieder.	Nicht in Quelle	`sudo dock` `compose s`
docker compose stop	Docker-Compose-Befehl	Hält die laufenden Container an, ohne sie zu entfernen.	Dienstname (optional)	`docker co` `stop`
docker compose exec	Docker-Compose-Befehl	Führt einen Befehl innerhalb eines laufenden Containers aus.	-it (interaktiv + TTY), Dienstname, Befehl	`docker co` `exec –it` `webserver`
docker compose logs	Docker-Compose-Befehl	Zeigt die Log-Ausgaben der Container an.	-f (follow), --tail (Zeilenanzahl)	`sudo dock` `compose l` `tail=50`
services	YAML-Konfigurationsschlüssel	Definiert den Block, in dem alle Container oder Dienste der Anwendung	Nicht in Quelle	`services:` `webserver` `image:` `nginx:1.2`

Befehl / Schlüsselwort	Kategorie	Beschreibung	Parameter / Optionen	Beispiel
		aufgelistet werden.		
image	YAML-Konfigurationsschlüssel	Gibt an, welches Docker-Image für den Dienst verwendet werden soll.	Image-Name und Version (Tag)	`image: nginx:1.2`
ports	YAML-Konfigurationsschlüssel	Definiert die Port-Bindungen zwischen dem Host und dem Container (entspricht -p).	Host-Port:Container-Port	`ports: - "8081:80"`
container_name	YAML-Konfigurationsschlüssel	Weist dem Container einen festen, lesbaren Namen zu (entspricht --name).	Name des Containers	`container_ mein-webs`
restart	YAML-Konfigurationsschlüssel	Legt die Neustart-Richtlinie fest, z. B. ob der Container nach einem Server-Neustart automatisch startet.	z. B. unless-stopped	`restart: stopped`